

Du texte à la connaissance structurée

Chapitre 7

Ce que trois jours de pipeline ont produit

- * Un corpus entièrement annoté : normalisé, POS, lemmatisé, entités nommées
- * C'est beaucoup — mais c'est encore du texte annoté, pas de la connaissance

Ce chapitre franchit la dernière étape : transformer ce texte annoté en structures interrogeables.

Trois opérations, deux formats de sortie

Modélisation thématique

Structure latente : quels sujets, dans quelles proportions

Extraction de relations

Liens sémantiques entre entités : qui agit sur qui, où, quand

Graphe de connaissances

Organise ces relations en structure interrogeable

TEI-XML et JSON-LD

Pérennité humanités numériques et web sémantique

Le point le plus important : la boucle NLP → HTR

- * Le pipeline NLP n'est pas seulement un consommateur de données HTR
- * Il devient un producteur de vérité terrain améliorée pour réentraîner le modèle HTR lui-même

Détaillé en partie 5 — c'est la fermeture du cycle complet du Volet 2.

PARTIE 1

Modélisation thématique

Trouver la structure sans supervision

- * Personne n'a écrit « ce document parle de fiscalité » sur le parchemin
- * La modélisation thématique infère des topics depuis la distribution des mots

Hypothèse fondamentale : chaque document est un mélange de topics, et chaque topic est une distribution sur le vocabulaire.

LDA : allocation de Dirichlet latente

Le modèle génératif fondateur (Blei et al. 2003)

- * Chaque document tire une distribution de topics θ
- * Chaque mot du document tire un topic, puis un mot selon ce topic

$$\Phi : K \text{ topics} \times V \text{ mots} \quad \text{—} \quad \Theta : D \text{ documents} \times K \text{ topics}$$

Les hyperparamètres α et β

α (alpha)

Concentration topic-document : petit α → chaque document dominé par peu de topics

β (beta)

Concentration mot-topic : petit β → chaque topic représenté par peu de mots

Configurer LDA en sklearn

```
lda = LatentDirichletAllocation(  
    n_components=10, max_iter=50,  
    doc_topic_prior=0.1, # alpha  
    topic_word_prior=0.01, # beta  
)  
lda.fit(X) # X : matrice document x vocabulaire
```

Vectorisation par sac de mots, avec une liste de stop words médiévaux (le, la, de, et...).

Thèmes typiques sur des chartes du XIVe siècle

Juridique

acte, charte, sceller, témoin, accord

Fiscal

denier, livre, taxe, bailli, compte

Nobiliaire

seigneur, chevalier, fief, vassal

Liturgique

abbaye, moine, dieu, prière, chapelle

Évaluer la qualité des topics : la cohérence c_v

* Corrèle la co-occurrence des mots de chaque topic dans un corpus de référence externe

> 0.5

Acceptable

0.6 – 0.7

Bon

NMF : une alternative matricielle

$$X \approx W \cdot H \quad (W, H \geq 0)$$

- * H : les K topics (distributions mot-topic), W : poids de chaque topic par document
- * Souvent plus rapide que LDA sur petits corpus, topics plus « purs »
- * Au prix d'une moins bonne base probabiliste

BERTopic : modélisation thématique neurale

Remplace le sac de mots par des embeddings contextuels

- * Étape 1 : embeddings SBERT/CamemBERT (768D) par document
- * Étape 2 : réduction de dimension par UMAP (vers ~5D)

UMAP préserve mieux la structure locale que PCA, ce qui produit des clusters plus séparés.

Étapes 3 et 4 : clustering et représentation

HDBSCAN

Pas besoin de fixer le nombre de clusters ;
identifie le bruit (label -1)

c-TF-IDF

Identifie les mots représentatifs de chaque
cluster/topic

Contrairement à K-Means, HDBSCAN n'impose pas qu'un document appartienne forcément à un topic.

Configurer BERTopic pour un corpus court

```
umap_model = UMAP(n_neighbors=10, n_components=5, metric='cosine')
hdbscan_model = HDBSCAN(min_cluster_size=5, min_samples=3)
topic_model = BERTopic(embedding_model=embedding_model,
                      umap_model=umap_model,
                      hdbscan_model=hdbscan_model)
topics, probs = topic_model.fit_transform(texts)
```

Pourquoi BERTopic sur un corpus court ?

- * Sur 200 chartes de ~80 tokens, les sacs de mots sont trop creux : LDA/NMF perdent en signal
- * Les embeddings SBERT capturent la sémantique même de phrases courtes

UMAP résout le problème de la malédiction de la dimensionnalité.

Le bruit n'est pas un défaut

- * HDBSCAN peut attribuer 30 à 40 % des documents au bruit (label -1) sur un corpus hétérogène

C'est une information : ces documents ne rentrent dans aucun topic dominant — souvent vrai pour les documents de contenu mixte.

PARTIE 2

Extraction de relations

Du texte au triplet

« *le sénéchal Jean de Normandie porta les lettres au roi* »

(Jean de Normandie, porte_titre, sénéchal)

(Jean de Normandie, effectue_action, porter les lettres)

(Jean de Normandie, destinataire, roi)

Chaque triplet (sujet, relation, objet) devient une arête du futur graphe de connaissances.

Approche par règles et patrons syntaxiques

- * Patrons sur l'annotation UD produite par Stanza
- * Exemple : TITLE suivi d'un nom propre composé → (PROPN+, porte_titre, TITLE)

```
# "sénéchal Jean de Normandie"  
TITLE PROPN+ ADP PROPN+ -> (PROPN+, porte_titre, TITLE)
```

Extraction par prompting LLM avec schéma

- * On fournit au LLM un schéma de relations cibles : porte_titre, réside_à, agit_lors_de, appartient_à, signe_acte_avec
- * Avantage : généralise au-delà des structures syntaxiques fixes des règles

« Jean, sénéchal de Normandie » et « Messire Jean de Gisors, qui tenait le titre de bailli » sont reconnus comme la même relation porte_titre.

Précision et validation

- * Précision typique d'un LLM bien guidé par schéma sur du moyen français normalisé : 75-85 %
- * La validation manuelle sur 50 triplets représentatifs suffit pour estimer la précision globale

Les triplets extraits par LLM nécessitent toujours une validation humaine sur un échantillon.

Résolution de coréférence

« *le roi* », « *Philippe* », « *il* », « *ledit seigneur* » : *un seul individu*

* Sans résolution, le graphe aurait plusieurs nœuds pour la même entité

Règles de base

« ledit seigneur » → dernière entité TITLE mentionnée

SpanBERT / coreferee

Pré-entraîné, mais pas spécifique au moyen français

LLM par prompting

Bon sur textes courts (<512 tokens), fenêtre glissante sinon

Coréférence par règles : exemple

- * « ledit / ladite » + TITLE/NOUN → coréfère à la dernière entité du type correspondant
- * « il / elle / lui » → coréfère au dernier nom propre de genre correspondant

La fonction maintient last_per, last_title, last_loc au fil du texte pour résoudre ces références.

PARTIE 3

Construction du graphe de connaissances

Structure d'un graphe de connaissances

- * Graphe orienté labellisé $G = (V, E)$
- * Nœuds V : entités (personnes, lieux, dates, organisations, titres)
- * Arêtes E : relations typées, directement issues des triplets extraits

NetworkX est la bibliothèque Python standard pour manipuler ces graphes.

Construire le graphe avec NetworkX

```
G = nx.DiGraph()
# Nœuds depuis les ner_spans
G.add_node(node_id, label=span['text'], type=span['label'],
            polygon=record['polygon_ref'])
# Arêtes depuis les relations extraites
G.add_edge(s_id, o_id, relation=rel['relation'],
            confiance=rel.get('confiance', 1.0))
```


Le lien avec le Volet 1

- * Chaque nœud porte un attribut polygon reprenant le polygon_ref du data contract

Depuis n'importe quelle requête sur le graphe, on remonte à la position physique de l'entité dans l'image du manuscrit — condition nécessaire pour la vérification par les humanistes.

Trois technologies de stockage

RDF/Turtle

Standard W3C — interopérabilité et archivage long terme (Wikidata, DBpedia)

Neo4j (Cypher)

Requêtes complexes en temps réel — pas de standard d'interopérabilité

JSON-LD

Le plus compact et accessible — un @context mappe les clés vers des URIs

RDF/Turtle : un exemple

```
medieval:jean_normandie a schema:Person ;  
  schema:name "Jean de Normandie" ;  
  medieval:porteTitre medieval:seneschal ;  
  medieval:residesAt medieval:normandie .
```

Chaque triplet (sujet, relation, objet) est encodé avec des URIs — lisible et interrogeable via SPARQL.

Choix pratique pour un projet de recherche

JSON-LD (échange/archivage) + Neo4j (exploration interactive)

RDF/Turtle est préféré quand l'interopérabilité avec d'autres bases du web sémantique est un objectif explicite.

Interroger le graphe : SPARQL, Cypher, RAG

SPARQL

Langage standard pour les graphes RDF

Cypher (Neo4j)

Plus expressif pour les traversées multi-niveaux

RAG

Interface en langage naturel :
index vectoriel + LLM

« Quels sénéchaux ont signé des actes en Normandie ? » — le RAG récupère les triplets pertinents et le LLM formule la réponse.

RAG minimal sur le graphe

- * 1. Extraire les entités de la question via le gazetier
- * 2. Récupérer le sous-graphe local (voisinage à 2 sauts)
- * 3. Formater les triplets comme contexte pour le LLM
- * 4. Générer la réponse, limitée aux triplets fournis

PARTIE 4

Export TEI-XML

TEI : le standard des humanités numériques

- * Encodage XML des textes en sciences humaines depuis 1987
- * Une centaine de balises : structure physique, structure logique, annotations linguistiques

Pour un projet de manuscrits médiévaux numériques, le TEI est la forme de publication scientifique attendue.

Structure minimale d'un document TEI

```
<TEI xmlns="http://www.tei-c.org/ns/1.0">
  <teiHeader>
    <fileDesc><titleStmt><title>Charte de Gisors, 1346</title>
    </titleStmt></fileDesc>
  </teiHeader>
  <text><body><div type="charte">
    <!-- contenu annoté -->
  </div></body></text>
</TEI>
```

Mapper les annotations NLP vers TEI

Annotation	Balise TEI	Attribut clé
PER	<persName>	@key
LOC	<placeName>	@key, @type
DATE	<date>	@when (ISO 8601)
ORG	<orgName>	@key
TITLE	<roleName>	@type="nobility"
Token POS+lemme	<w>	@lemma, @pos

L'ancrage spatial : `<lb/>` et `@facs`

- * `<lb/>` (line break) marque une ligne manuscrite
- * `@facs` reprend le `polygon_ref` du data contract Volet 1

@facs réalise la connexion entre l'annotation linguistique et la position physique sur le parchemin.

Générer le TEI avec lxml

```
lb = etree.SubElement(parent, f'{{{TEI_NS}}}{lb}',  
                       n=record['line_id'],  
                       facs=record['polygon_ref'])  
# Pour chaque token : <w lemma="..." pos="...">forme</w>  
# Entités encadrées par <persName key="...">, <placeName>, etc.
```

Valider la conformité TEI

```
wget .../tei_all.rng  
xmllint --relaxng tei_all.rng --noout corpus_medieval.xml  
# corpus_medieval.xml validates
```

La validation contre le schéma RelaxNG officiel garantit l'interopérabilité avec les outils TEI existants.

PARTIE 5

La boucle de rétroaction NLP → HTR

Le principe : un chemin de retour

- * Le pipeline va de l'image au graphe : HTR → normalisation → NER → graphe
- * Mais les corrections NLP constituent une nouvelle vérité terrain pour le modèle HTR

La plupart des projets ignorent ce chemin de retour.

Le mécanisme

Transcription HTR : "norm~die" → Forme normalisée : "normandie"

Cette paire (transcription, forme corrigée) est exactement la donnée d'entraînement dont le HTR a besoin pour améliorer son comportement sur les abréviations.

Trois critères pour retenir une correction

- * Confiance HTR suffisante (ligne bien reconnue globalement)
- * Une correction existe réellement (transcription $\neq$ normalisée)
- * La correction est non triviale (CER de la correction $>$ seuil minimal)

Classer le type de correction

abréviation

Présence de ~, ^ ou ñ dans la transcription HTR

graphie_uv

u/v dans le HTR, résolu différemment dans la forme normalisée

autre

Toute autre correction

Cette typologie aide à comprendre quelles erreurs HTR sont effectivement corrigées par la boucle.

Le risque : le biais de confirmation

- * Si une règle de `apply_grapheme_rules` est incorrecte, elle normalise une forme fautive en une autre forme fautive
- * Injectée dans l'entraînement HTR, cette erreur s'ancre dans le nouveau modèle

Le modèle HTR réentraîné produit davantage cette forme erronée, qui sera de nouveau normalisée de la même façon — une boucle de dégradation auto-entretenu.

Trois garde-fous indispensables

Validation humaine

10 % minimum des corrections, avant injection

split_hash respecté

Interdiction d'injecter des corrections issues du jeu de test HTR

Comparaison CER avant/après

Sur un jeu de test indépendant — seule validation objective

Si le CER sur le test ne s'améliore pas, les corrections n'ont pas été bénéfiques — même si les métriques NLP s'améliorent.

PARTIE 6

Synthèse : le data contract NLP final

L'artefact de sortie du Volet 2

Tous les enrichissements depuis le Jour 1

```
{
  "line_id": "charte_1346_fol12_103", "polygon_ref": "fol12_bbox_230...",
  "normalized": "li seigneur de gisors signa",
  "ner_spans": [...], "pos_tags": [...], "lemmas": [...],
  "topics": [{"topic_id": 2, "label": "juridique", "weight": 0.72}],
  "relations": [{"sujet": "seigneur_gisors", "relation": "réside_à",
    "objet": "gisors", "confiance": 0.91 }],
  "graph_node_id": "medieval:seigneur_gisors",
  "tei_ref": "corpus_medieval.xml#charte_1346_fol12_103"
}
```

Ce que ce contrat rend possible : interrogation

- * Poser une question en SPARQL ou Cypher (« quels seigneurs ont signé des actes à Gisors entre 1340 et 1360 ? »)
- * Obtenir une réponse avec références précises aux documents

Traçabilité et confiance

Ce que ce contrat rend possible

Traçabilité

Suivre tout résultat jusqu'à l'image source via polygon_ref

Fiabilité

Évaluer chaque annotation via confidence et les scores de confiance des relations

Réutilisabilité

Formats ouverts et standards — TEI ou JSON-LD, compatibles avec tout outil

Pour aller plus loin

Modélisation thématique et extraction de relations

Topics

Blei et al. (2003), LDA ; Lee & Seung (1999), NMF ; Grootendorst (2022), BERTopic

Relations & corréférence

Joshi et al. (2020), SpanBERT ; Mintz et al. (2009), distant supervision

Pour aller plus loin

Graphes, TEI et apprentissage actif

Graphes et web sémantique

Bizer et al. (2009), Linked Data ; W3C SPARQL et JSON-LD

TEI

TEI Consortium (2023), Guidelines P5 ; Burnard (2014)

Boucle de rétroaction

Settles (2012), Active Learning ; Kiela et al. (2021), Dynabench